



4

Integrating a Frontend Using React and TanStack Query

After designing, implementing, and testing our backend service, it's now time to create a frontend to interface with the backend. First, we will start by setting up a full-stack React project based on the Vite boilerplate and the backend service created in the previous chapters. Then, we are going to create a basic user interface for our blog application. Finally, we will use TanStack Query, a data fetching library to handle backend state, to integrate the backend API into the frontend. By the end of this chapter, we will have successfully developed our first full-stack application!

In this chapter, we are going to cover the following main topics:

- Principles of React
- Setting up a full-stack React project
- Creating the user interface for our application
- Integrating the backend service using TanStack Query

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing for Full-stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in [*Chapter 1*](#) and 2.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch4>

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA video for this chapter can be found at:

<https://youtu.be/WXqJu2Ut7Hs>

Principles of React

Before we start learning how to set up a full-stack React project, let's revisit the three fundamental principles of React. These principles allow us to easily write scalable web applications:

- **Declarative:** Instead of telling React how to do things, we tell it what we want it to do. As a result, we can easily design our applications and React will efficiently update and render just the right components when the data changes. For example, the following code, which duplicates strings in an array is imperative, which is the opposite of declarative:

```
const input = ['a', 'b', 'c']
let result = []
for (let i = 0; i < input.length; i++) {
  result.push(input[i] + input[i])
}
```

```
}  
console.log(result) // prints: [ 'aa', 'bb', 'cc' ]
```

As we can see, in imperative code, we need to tell JavaScript exactly what to do, step by step. However, with declarative code, we can simply tell the computer what we want, as follows:

```
const input = ['a', 'b', 'c']  
const result = input.map(str => str + str)  
console.log(result) // prints: ['aa', 'bb', 'cc']
```

In this declarative code, we tell the computer that we want to map each element of the **input** array from **str** to **str + str**. As you can see, declarative code is much more concise.

- **Component-based:** React encapsulates components that manage their own state and views and then allows us to compose them in order to create complex user interfaces.
- **Learn once, write anywhere:** React does not make assumptions about your technology stack and tries to ensure that you can develop apps without rewriting existing code as much as possible.

React's three fundamental principles make it easy to write code, encapsulate components, and share code across multiple platforms. Instead of reinventing the wheel, React tries to make use of existing JavaScript features as much as possible. As a result, we will learn software design patterns that will be applicable in many more cases than just designing user interfaces.

Now that we have learned the fundamental principles of React, let's get started setting up a full-stack React project!

Setting up a full-stack React project

Before we can start developing our frontend application, we first need to merge our previously created frontend boilerplate based on Vite with the backend service created in ***Chapter 3, Implementing a Backend***

Using Express, Mongoose ODM, and Jest. Let's merge them now by following these steps:

1. Copy the **ch1** folder to a new **ch4** folder, as follows:

```
$ cp -R ch1 ch4
```

2. Copy the **ch3** folder to a new **ch4/backend** folder, as follows:

```
$ cp -R ch3 ch4/backend
```

3. Delete the **.git** folder in the copied **ch4/backend** folder, as follows:

```
$ rm -rf ch4/backend/.git
```

4. Open the new **ch4** folder in VS Code.
5. Remove the Husky **prepare** script (the line is highlighted in the code snippet) from the **backend/package.json** file, as we already have Husky set up in the root directory:

```
"scripts": {  
  "dev": "nodemon src/index.js",  
  "start": "node src/index.js",  
  "test": "NODE_OPTIONS=--experimental-vm-modules jest",  
  "lint": "eslint src",  
  "prepare": "husky install"  
},
```

6. Also remove the following **lint-staged** config from the **backend/package.json** file:

```
"lint-staged": {  
  "**/*.js,jsx": [  
    "npx prettier --write",  
    "npx eslint --fix"  
  ]  
}
```

7. Then, *remove* the **backend/.husky**, **backend/.vscode**, and **backend/.git** folders.
8. To make sure all dependencies are installed properly, run the following command in the root of the **ch4** folder:

```
$ npm install
```

9. Also go to the **backend/** directory and install all dependencies there:

```
$ cd backend/  
$ npm install
```

10. We can now also remove the **husky**, **lint-staged**, and **@commitlint** packages from the backend project, as we already have it set up in the main project folder:

```
$ npm uninstall husky lint-staged \  
@commitlint/cli @commitlint/config-conventional
```

TIP

It is always a good idea to regularly check which packages you still need and which you can get rid of, to keep your project clean. In this case, we copied code from another project, but do not need the Husky / lint-staged / commitlint setup, as we already have it set up in the root of our project.

11. Now go back to the root of the **ch4** folder and run the following command to start the frontend server:

```
$ cd ../  
$ npm run dev
```

12. Open the frontend in your browser by going to the URL shown by Vite: **http://localhost:5173/**
13. Open **src/App.jsx**, change the title as follows, and save the file:

```
<h1>Vite + React + Node.js</h1>
```

14. You will see that the change is reflected instantly in the browser!

After successfully setting up our full-stack project by combining our projects from previous chapters, let's now get started designing and creating the user interface for our blog application.

Creating the user interface for our application

When designing the structure of a frontend, we should also consider the folder structure, so that our app can grow easily in the future. Similar to how we did for the backend, we will also put all our source code into a **src/** folder. We can then group the files in separate folders for the different features. Another popular way to structure frontend projects is to group code by routes. Of course, it is also possible to mix them, for example, in Next.js projects we can group our components by features and then create another folder and file structure for the routes, where the components are used. For full-stack projects, it additionally makes sense to first separate our code by creating separate folders for the API integration and UI components.

Now, let's define the folder structure for our project:

1. Create a new **src/api/** folder.
2. Create a new **src/components/** folder.

TIP

It is a good idea to start with a simple structure at first, and only nest more deeply when you actually need it. Do not spend too much time thinking about the file structure when starting a project, because usually,

you do not know upfront how files should be grouped, and it may change later anyway.

After defining the high-level folder structure for our projects, let's now take some time to consider the component structure.

Component structure

Based on what we defined in the backend, our blog application is going to have the following features:

- Viewing a single post
- Creating a new post
- Listing posts
- Filtering posts
- Sorting posts

The idea of components in React is to have each component deal with a single task or UI element. We should try to make components as fine-grained as possible, in order to be able to reuse code. If we find ourselves copying and pasting code from one component to another, it might be a good idea to create a new component and reuse it in multiple other components.

Usually, when developing a frontend, we start with a UI mock-up. For our blog application, a mock-up could look as follows:

Title:

Author:

Create

author:

Sort By: / Sort Order:

Full-Stack React Projects

Let's become full-stack developers!

*Written by **Daniel Bugl***

Hello React!

Figure 4.1 – An initial mock-up of our blog application

NOTE

In this book, we will not cover UI or CSS frameworks. As such, the components are designed and developed without styling. Instead, the book focuses on the full-stack aspect of the integration of backends with front-ends. Feel free to use a UI framework (such as MUI), or a CSS framework (such as Tailwind) to style the blog application on your own.

When splitting up the UI into components, we use the **single-responsibility principle**, which states that every module should have responsibility over a single encapsulated part of the functionality.

In our mock-up, we can draw boxes around each component and sub-component, and give them names. Keep in mind that each component should have exactly one responsibility. We start with the fundamental components that make up the app:

The mock-up displays five distinct UI components, each enclosed in a colored border and labeled to the right:

- CreatePost** (green border): A form with fields for 'Title:', 'Author:', and a larger text area, followed by a 'Create' button.
- PostFilter** (blue border): A section with 'Filter by:' and an 'author:' input field.
- PostSorting** (purple border): A section with 'Sort By:' (showing 'createdAt') and 'Sort Order:' (showing 'descending'), both with dropdown menus.
- Post** (red border): A post card titled 'Full-Stack React Projects' with the text 'Let's become full-stack developers!' and 'Written by *Daniel Bugl*'.
- Post** (red border): A post card titled 'Hello React!'.

Figure 4.2 – Defining the fundamental components in our mock-up

We defined a **CreatePost** component, with a form to create a new post, a **PostFilter** component to filter the list of posts, a **PostSorting** component to sort posts, and a **Post** component to display a single post.

Now that we have defined our fundamental components, we are going to look at which components logically belong together, thereby forming a group: we can group the **Post** components together in **PostList**, then make an **App** component to group everything together and define the structure of our app.

Now that we are done with structuring our React components, we can move on to implementing the static React components.

Implementing static React components

Before integrating with the backend, we are going to model the basic features of our application as static React components. Dealing with the static view structure of our application first makes sense, as we can play around and re-structure the application UI if needed, before adding integration to the components, which would make it harder and more tedious to move them around. It is also easier to deal only with the UI first, which helps us to get started quickly with projects and features. Then, we can move on to implementing integrations and handling state.

Let's get started implementing the static components now.

The Post component

We have already thought about which elements a post has during the creation of the mock-up and the design of the backend. A post should have a **title**, **contents**, and an **author**.

Let's implement the **Post** component now:

1. First, create a new **src/components/Post.jsx** file.
2. In that file, import **PropTypes**:

```
import PropTypes from 'prop-types'
```

3. Define a function component, accepting **title**, **contents**, and **author** props:

```
export function Post({ title, contents, author }) {
```

4. Next, render all props in a way that resembles the mock-up:

```
  return (  
    <article>  
      <h3>{title}</h3>  
      <div>{contents}</div>  
      {author && (  
        <em>  
          <br />  
          Written by <strong>{author}</strong>  
        </em>  
      )}  
    </article>  
  )  
}
```

TIP

Please note that you should always prefer spacing via CSS, rather than using the `<br />` HTML tag. However, we are focusing on the UI structure and integration with the backend in this book, so we simply use HTML whenever possible.

5. Now, define **propTypes**, making sure only **title** is required:

```
Post.propTypes = {  
  title: PropTypes.string.isRequired,  
  contents: PropTypes.string,  
  author: PropTypes.string,  
}
```

INFO

PropTypes are used to validate the props passed to React components and to ensure that we are passing the correct props when using JavaScript. When using a type-safe language, such as TypeScript, we can instead do this by directly typing the props passed to the component.

6. Let's test out our component by replacing the **src/App.jsx** file with the following contents:

```
import { Post } from './components/Post.jsx'
export function App() {
  return (
    <Post
      title='Full-Stack React Projects'
      contents="Let's become full-stack developers!"
      author='Daniel Bugl'
    />
  )
}
```

7. Edit **src/main.jsx** and update the import of the **App** component, because we are now not using **export default** anymore:

```
import { App } from './App.jsx'
```

INFO

I personally tend to prefer not using default exports, as they make it harder to re-group and re-export components and functions from other files. Also, they allow us to change the names of the components, which could be confusing. For example, if we change the name of a component, the name when importing it is not changed automatically.

8. Also, remove the following line from **src/main.jsx**:

```
import './index.css'
```

9. Finally, we can *delete* the **index.css** and **App.css** files, as they are not needed anymore.

Now that our static **Post** component has been implemented, we can move on to the **CreatePost** component.

The CreatePost component

We'll now implement a form to allow for the creation of new posts. Here, we provide fields for **author** and **title** and a `<textarea>` element for the contents of the blog post.

Let's implement the **CreatePost** component now:

1. Create a new **src/components/CreatePost.jsx** file.
2. Define the following component, which contains a form to enter the title, author, and contents of a blog post:

```
export function CreatePost() {
  return (
    <form onSubmit={(e) => e.preventDefault()}>
      <div>
        <label htmlFor='create-title'>Title: </label>
        <input type='text' name='create-title' id='create-title' />
      </div>
      <br />
      <div>
        <label htmlFor='create-author'>Author: </label>
        <input type='text' name='create-author' id='create-author' />
      </div>
      <br />
      <textarea />
      <br />
      <br />
      <input type='submit' value='Create' />
    </form>
  )
}
```

In the preceding code block, we defined an **onSubmit** handler and called **e.preventDefault()** on the event object to avoid a page refresh when the form is submitted.

3. Let's test the component out by *replacing* the **src/App.jsx** file with the following contents:

```
import { CreatePost } from './components/CreatePost.jsx'
export function App() {
  return <CreatePost />
}
```

As you can see, the **CreatePost** component renders fine. We can now move on to the **PostFilter** and **PostSorting** components.

TIP

If you want to test out multiple components at once and keep the tests around for later, or build a style guide for your own component library, you should look into Storybook (<https://storybook.js.org>), which is a useful tool to build, test, and document UI components in isolation.

The **PostFilter** and **PostSorting** components

Similar to the **CreatePost** component, we will be creating two components that provide input fields to filter and sort posts. Let's start with **PostFilter**:

1. Create a new **src/components/PostFilter.jsx** file.
2. In this file, we import **PropTypes**:

```
import PropTypes from 'prop-types'
```

3. Now, we define the **PostFilter** component and make use of the **field** prop:

```
export function PostFilter({ field }) {
```

```

return (
  <div>
    <label htmlFor={`filter-${field}`}>{field}: </label>
    <input
      type='text'
      name={`filter-${field}`}
      id={`filter-${field}`}
    />
  </div>
)
}
PostFilter.propTypes = {
  field: PropTypes.string.isRequired,
}

```

Next, we are going to define the **PostSorting** component.

4. Create a new **src/components/PostSorting.jsx** file.
5. In this file, we create a **select** input to select which field to sort by.

We also create another **select** input to select the sort order:

```

import PropTypes from 'prop-types'
export function PostSorting({ fields = [] }) {
  return (
    <div>
      <label htmlFor='sortBy'>Sort By: </label>
      <select name='sortBy' id='sortBy'>
        {fields.map((field) => (
          <option key={field} value={field}>
            {field}
          </option>
        ))}
      </select>
      {' / '}
      <label htmlFor='sortOrder'>Sort Order: </label>
      <select name='sortOrder' id='sortOrder'>
        <option value='ascending'>ascending</option>
        <option value='descending'>descending</option>
      </select>
    </div>
  )
}

```

```
}  
PostSorting.propTypes = {  
  fields: PropTypes.arrayOf(PropTypes.string).isRequired,  
}
```

Now we have successfully defined UI components to filter and sort posts. In the next step, we are going to create a **PostList** component to combine the filter and sorting with a list of posts.

The PostList component

After implementing the other post-related components, we can now implement the most important part of our blog app, that is, the feed of blog posts. For now, the feed is simply going to show a list of blog posts.

Let's start implementing the **PostList** component now:

1. Create a new **src/components/PostList.jsx** file.
2. First, we import **Fragment**, **PropTypes**, and the **Post** component:

```
import { Fragment } from 'react'  
import PropTypes from 'prop-types'  
import { Post } from './Post.jsx'
```

3. Then, we define the **PostList** function component, accepting a **posts** array as a prop. If **posts** is not defined, we set it to an empty array, by default:

```
export function PostList({ posts = [] }) {
```

4. Next, we render all posts by using the **.map** function and the spread syntax:

```
  return (  
    <div>  
      {posts.map((post) => (  
        <Post {...post} key={post._id} />  
      )
```



```
    )})  
  </div>  
)  
}
```

We return the **<Post>** component for each post, and pass all the keys from the **post** object to the component as props. We do this by using the spread syntax, which has the same effect as listing all the keys from the object manually as props, like so:

```
<Post  
  title={post.title}  
  author={post.author}  
  contents={post.contents}  
/>
```

NOTE

*If we are rendering a list of elements, we have to give each element a unique **key** prop. React uses this **key** prop to efficiently compute the difference between two lists when the data has changed.*

We used the **map** function, which applies a function to all the elements of an array. This is similar to using a **for** loop and storing all the results, but it is more concise, declarative, and easier to read! Alternatively, we could do the following instead of using the **map** function:

```
let renderedPosts = []  
let index = 0  
for (let post of posts) {  
  renderedPosts.push(<Post {...post} key={post._id} />)  
  index++  
}  
return (  
  <div>  
    {renderedPosts}
```

```
    </div>  
  )
```

However, using this style is *not* recommended with React.

5. We also still need to define the prop types. Here, we can make use of the prop types from the **Post** component, by wrapping it inside the **PropTypes.shape()** function, which defines an object prop type:

```
PostList.propTypes = {  
  posts: PropTypes.arrayOf(PropTypes.shape(Post.propTypes)).isRequired,  
}
```

6. In the mock-up, we have a horizontal line after each blog post. We can implement this without an additional **<div>** container element, by using **Fragment**, as follows:

```
    {posts.map((post) => (  
      <Fragment key={post._id}>  
        <Post {...post} />  
        <hr />  
      </Fragment>  
    ))}
```

NOTE

*The **key** prop always has to be added to the uppermost parent element that is rendered within the **map** function. In this case, we had to move the **key** prop from the **Post** component to **Fragment**.*

7. Again, we test our component by editing the **src/App.jsx** file:

```
import { PostList } from '../components/PostList.jsx'  
const posts = [  
  {  
    title: 'Full-Stack React Projects',  
    contents: "Let's become full-stack developers!",  
    author: 'Daniel Bugl',
```

```

    },
    { title: 'Hello React!' },
  ]
  export function App() {
    return <PostList posts={posts} />
  }

```

Now we can see that our app lists all the posts that we defined in the **posts** array.

As you can see, listing multiple posts via the **PostList** component works fine. We can now move on to putting the app together.

Putting the app together

After implementing all the components, we now have to put everything together in the **App** component. Then, we will have successfully reproduced the mock-up!

Let's start modifying the **App** component and putting our blog app together:

1. Open **src/App.jsx** and add imports for the **CreatePost**, **PostFilter**, and **PostSorting** components:

```

import { PostList } from './components/PostList.jsx'
import { CreatePost } from './components/CreatePost.jsx'
import { PostFilter } from './components/PostFilter.jsx'
import { PostSorting } from './components/PostSorting.jsx'

```

2. Adjust the **App** component to contain all the components:

```

export function App() {
  return (
    <div style={{ padding: 8 }}>
      <CreatePost />
      <br />
      <hr />
      Filter by:

```

```
    <PostFilter field='author' />
    <br />
    <PostSorting fields={['createdAt', 'updatedAt']} />
    <hr />
    <PostList posts={posts} />
  </div>
)
}
```

3. After saving the file, the browser should automatically refresh, and we can now see the full UI:



Figure 4.3 – Full implementation of our static blog app, according to the mock-up

As we can see, all of the static components that we defined earlier are rendered together in one **App** component. Our app now looks just like a

mock-up. Next, we can move on to integrating our components with the backend service.

Integrating the backend service using TanStack Query

After finishing creating all the UI components, we can now move on to integrating them with the backend we created in the previous chapter. For the integration, we are going to use TanStack Query (previously called React Query), which is a data fetching library that can also help us with caching, synchronizing, and updating data from a backend.

TanStack Query specifically focuses on managing the state of fetched data (server state). While other state management libraries can also deal with server state, they specialize in managing client state instead. Server state has some stark differences from client state, such as the following:

- Being persisted remotely in a location the client does not control directly
- Requiring asynchronous APIs to fetch and update state
- Having to deal with shared ownership, which means that other people can change the state without your knowledge
- State becoming stale (“out of date”) at some point when changed by the server or other people

These challenges with server state result in issues such as having to cache, deduplicate multiple requests, update “out of date” state in the background, and so on.

TanStack Query provides solutions to these issues out of the box and thus makes dealing with server state simple. You can always combine it with other state management libraries that focus on client state as well. For use cases where the client state essentially just reflects the server

state though, TanStack Query on its own can be good enough as a state management solution!

NOTE

The reason why React Query got renamed to TanStack Query is that the library now also supports other frameworks, such as Solid, Vue, and Svelte!

Now that you know why and how TanStack Query can help us integrate our frontend with the backend, let's get started using it!

Setting up TanStack Query for React

To set up TanStack Query, we first have to install the dependency and set up a query client. The query client is provided to React through a context and will store information about active requests, cached results, when to periodically re-fetch data, and everything needed for TanStack Query to function.

Let's get started setting it up now:

1. Open a new Terminal (do not quit Vite!) and install the **@tanstack/react-query** dependency by running the following command in the root of our project:

```
$ npm install @tanstack/react-query@5.12.2
```

We are now going to move our current **App** component to a new **Blog** component, as we are going to use the **App** component for setting up libraries and contexts instead.

2. Rename the **src/App.jsx** file to **src/Blog.jsx**.
Do not update imports yet. If VS Code asks you to update imports, click **No**.
3. Now, in **src/Blog.jsx**, change the function name from **App** to **Blog**:

```
export function Blog() {
```

4. Create a new **src/App.jsx** file. In this file, import **QueryClient** and **QueryClientProvider** from TanStack React Query:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
```

5. Also, import the **Blog** component:

```
import { Blog } from './Blog.jsx'
```

6. Now, create a new query client:

```
const queryClient = new QueryClient()
```

7. Define the **App** component and render the **Blog** component wrapped inside **QueryClientProvider**:

```
export function App() {  
  return (  
    <QueryClientProvider client={queryClient}>  
      <Blog />  
    </QueryClientProvider>  
  )  
}
```

That's all there is to setting up TanStack Query! We can now make use of it inside our **Blog** component (and its children).

Fetching blog posts

The first thing we should do is fetch the list of blog posts from our backend. Let's implement that now:

1. First of all, in the second Terminal window opened (not where Vite is running), run the backend server (do not quit Vite!), as follows:

```
$ cd backend/  
$ npm start
```

If you get an error, make sure Docker and MongoDB are running properly!

TIP

If you want to develop the backend and frontend at the same time, you can start the backend using `npm run dev` to make sure it hot reloads when you change the code.

2. Create a `.env` file in the root of the project, and enter the following contents into it:

```
VITE_BACKEND_URL="http://localhost:3001/api/v1"
```

Vite supports **dotenv** out of the box. All environment variables that should be available to be accessed within the frontend need to be prefixed with **VITE_**. Here, we set an environment variable to point to our backend server.

3. Create a new `src/api/posts.js` file. In this file, we are going to define a function to fetch posts, which accepts the query params for the `/posts` endpoint as an argument. These query params are used to filter by author and tag and define sorting using **sortBy** and **sortOrder**:

```
export const getPosts = async (queryParams) => {
```

4. Remember that we can use the **fetch** function to make a request to a server. We need to pass the environment variable to it and add the `/posts` endpoint. After the path, we add query params, which are prefixed with the `?` symbol:

```
const res = await fetch(  

```



```
`${import.meta.env.VITE_BACKEND_URL}/posts?` +
```

5. Now we need to use the **URLSearchParams** class to turn an object into query params. That class will automatically escape the input for us and turn it into valid query params:

```
new URLSearchParams(queryParams),
```

6. Like we did before in the browser, we need to parse the response as JSON:

```
)  
return await res.json()  
}
```

7. Edit **src/Blog.jsx** and *remove* the sample **posts** array:

```
const posts = [  
  {  
    title: 'Full-Stack React Projects',  
    contents: "Let's become full-stack developers!",  
    author: 'Daniel Bugl',  
  },  
  { title: 'Hello React!' },  
]
```

8. Also, import the **useQuery** function from **@tanstack/react-query** and the **getPosts** function from our **api** folder in the **src/Blog.jsx** file:

```
import { useQuery } from '@tanstack/react-query'  
import { PostList } from './components/PostList.jsx'  
import { CreatePost } from './components/CreatePost.jsx'  
import { PostFilter } from './components/PostFilter.jsx'  
import { PostSorting } from './components/PostSorting.jsx'  
import { getPosts } from './api/posts.js'
```

9. Inside the **Blog** component, define a **useQuery** hook:

```
export function Blog() {  
  const postsQuery = useQuery({  
    queryKey: ['posts'],  
    queryFn: () => getPosts(),  
  })
```

The **queryKey** is very important in TanStack Query, as it is used to uniquely identify a request, among other things, for caching purposes. Always make sure to use unique query keys. Otherwise, you might see requests not triggering properly.

For the **queryFn** option, we just call the **getPosts** function, without query params for now.

10. After the **useQuery** hook, we get the posts from our query and fall back to an empty array if the posts are not loaded yet:

```
const posts = postsQuery.data ?? []
```

11. Check your browser, and you will see that the posts are now loaded from our backend!

Now that we have successfully fetched blog posts, let's get the filters and sorting working!

Implementing filters and sorting

To implement filters and sorting, we need to handle some local state and pass it as query params to **postsQuery**. Let's do that now:

1. We start by editing the **src/Blog.jsx** file and importing the **useState** hook from React:

```
import { useState } from 'react'
```

2. Then we add state hooks for the **author** filter and the sorting options inside the **Blog** component, before the **useQuery** hook:

```
const [author, setAuthor] = useState('')
const [sortBy, setSortBy] = useState('createdAt')
const [sortOrder, setSortOrder] = useState('descending')
```

3. Then, we adjust **queryKey** to contain the query params (so that whenever a query param changes, TanStack Query will re-fetch unless the request is already cached). We also adjust **queryFn** to call **getPosts** with the relevant query params:

```
const postsQuery = useQuery({
  queryKey: ['posts', { author, sortBy, sortOrder }],
  queryFn: () => getPosts({ author, sortBy, sortOrder }),
})
```

4. Now pass the values and relevant **onChange** handlers to the filter and sorting components:

```
<PostFilter
  field='author'
  value={author}
  onChange={(value) => setAuthor(value)}
/>
<br />
<PostSorting
  fields={['createdAt', 'updatedAt']}
  value={sortBy}
  onChange={(value) => setSortBy(value)}
  orderValue={sortOrder}
  onOrderChange={(orderValue) => setSortOrder(orderValue)}
/>
```

NOTE

For simplicity's sake, we are only using state hooks for now. A state management solution or context could make dealing with filters and sorting much easier, especially for larger applications. For our small blog appli-

cation, it is fine to use state hooks though, as we are focusing mostly on the integration of the backend and frontend.

5. Now, edit `src/components/PostFilter.jsx` and add the `value` and `onChange` props:

```
export function PostFilter({ field, value, onChange }) {
  return (
    <div>
      <label htmlFor={`filter-${field}`}>{field}: </label>
      <input
        type='text'
        name={`filter-${field}`}
        id={`filter-${field}`}
        value={value}
        onChange={(e) => onChange(e.target.value)}
      />
    </div>
  )
}
PostFilter.propTypes = {
  field: PropTypes.string.isRequired,
  value: PropTypes.string.isRequired,
  onChange: PropTypes.func.isRequired,
}
```

6. We also do the same for `src/components/PostSorting.jsx`:

```
export function PostSorting({
  fields = [],
  value,
  onChange,
  orderValue,
  onOrderChange,
}) {
  return (
    <div>
      <label htmlFor='sortBy'>Sort By: </label>
      <select
        name='sortBy'

```

```

      id='sortBy'
      value={value}
      onChange={(e) => onChange(e.target.value)}
    >
      {fields.map((field) => (
        <option key={field} value={field}>
          {field}
        </option>
      ))}
    </select>
    {' / '}
    <label htmlFor='sortOrder'>Sort Order: </label>
    <select
      name='sortOrder'
      id='sortOrder'
      value={orderValue}
      onChange={(e) => onOrderChange(e.target.value)}
    >
      <option value='ascending'>ascending</option>
      <option value='descending'>descending</option>
    </select>
  </div>
)
}
PostSorting.propTypes = {
  fields: PropTypes.arrayOf(PropTypes.string).isRequired,
  value: PropTypes.string.isRequired,
  onChange: PropTypes.func.isRequired,
  orderValue: PropTypes.string.isRequired,
  onOrderChange: PropTypes.func.isRequired,
}

```

7. In your browser, enter **Daniel Bugl** as the author. You should see TanStack Query re-fetch the posts from the backend as you type, and once a match is found, the backend will return all posts by that author!
8. After testing it out, make sure to clear the filter again, so that newly created posts are not filtered by the author anymore later on.

TIP

*If you do not want to make that many requests to the backend, make sure to use a debouncing state hook, such as **useDebounce**, and then pass only the debounced value to the query param. If you are interested in gaining further knowledge about the **useDebounce** hook and other useful hooks, I recommend checking out my book titled *Learn React Hooks*.*

The application should now look as follows, with the posts being filtered by the author entered in the field, and sorted by the selected field, in the selected order:

Title:

Author:

Create

Filter by:

author:

Sort By: / Sort Order:

Hello Mongoose!

Written by Daniel Bugl

Hello again, Mongoose!

Written by Daniel Bugl

Figure 4.4 – Our first full-stack application – a frontend fetching posts from a backend!

Now that sorting and filtering are working properly, let's learn about mutations, which allow us to make requests to the server that change the state of the backend (for example, inserting or updating entries in the database).

Creating new posts

We are now going to implement a feature to create posts. To do this, we need to use the **useMutation** hook from TanStack Query. While queries are meant to be idempotent (meaning that calling them multiple times should not affect the result), mutations are used to create/update/delete data or perform operations on the server. Let's get started using mutations to create new posts now:

1. Edit **src/api/posts.js** and define a new **createPost** function, which accepts a **post** object as an argument:

```
export const createPost = async (post) => {
```

2. We also make a request to the **/posts** endpoint, like we did for **getPosts**:

```
const res = await fetch(`${import.meta.env.VITE_BACKEND_URL}/posts`, {
```

3. However, now we also set **method** to a **POST** request, pass a header to tell the backend that we will be sending a JSON body, and then send our **post** object as a JSON string:

```
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify(post),
```

4. Like with **getPosts**, we also need to parse the response as JSON:

```
  })
```



```
    return await res.json()
  }
```

After defining the **createPost** API function, let's use it in the **CreatePost** component by creating a new mutation hook there.

5. Edit **src/components/CreatePost.jsx** and import the **useMutation** hook from **@tanstack/react-query**, the **useState** hook from **React**, and our **createPost** API function:

```
import { useMutation } from '@tanstack/react-query'
import { useState } from 'react'
import { createPost } from '../api/posts.js'
```

6. Inside the **CreatePost** component, define state hooks for **title**, **author**, and **contents**:

```
const [title, setTitle] = useState('')
const [author, setAuthor] = useState('')
const [contents, setContents] = useState('')
```

7. Now, define a mutation hook. Here, we are going to call our **createPost** function:

```
const createPostMutation = useMutation({
  mutationFn: () => createPost({ title, author, contents }),
})
```

8. Next, we are going to define a **handleSubmit** function, which will prevent the default submit action (which refreshes the page), and instead call **.mutate()** to execute the mutation:

```
const handleSubmit = (e) => {
  e.preventDefault()
  createPostMutation.mutate()
}
```

9. We add the **onSubmit** handler to our form:

```
<form onSubmit={handleSubmit}>
```

10. We also add the **value** and **onChange** props to our fields, as we did before for the sorting and filters:

```
<div>
  <label htmlFor='create-title'>Title: </label>
  <input
    type='text'
    name='create-title'
    id='create-title'
    value={title}
    onChange={(e) => setTitle(e.target.value)}
  />
</div>
<br />
<div>
  <label htmlFor='create-author'>Author: </label>
  <input
    type='text'
    name='create-author'
    id='create-author'
    value={author}
    onChange={(e) => setAuthor(e.target.value)}
  />
</div>
<br />
<textarea
  value={contents}
  onChange={(e) => setContents(e.target.value)}
/>
```

11. For the submit button, we make sure it says **Creating...** instead of **Create** while we are waiting for the mutation to finish, and we also disable the button if no title was set (as it is required), or if the mutation is currently pending:

```
<br />
```

```
<br />
<input
  type='submit'
  value={createPostMutation.isPending ? 'Creating...' : 'Create'}
  disabled={!title || createPostMutation.isPending}
/>
```

12. Lastly, we add a message below the submit button, which will be shown if the mutation is successful:

```
{createPostMutation.isSuccess ? (
  <>
    <br />
    Post created successfully!
  </>
) : null}
</form>
```

NOTE

*In addition to **isPending** and **isSuccess**, mutations also return **isIdle** (when the mutation is idle or in a fresh/reset state) and **isError** states. The same states can also be accessed from queries, for example, to show a loading animation while posts are fetching.*

13. Now we can try adding a new post, and it seems to work fine, but the post list is not updating automatically, only after a refresh!

The issue is that the query key did not change, so TanStack Query does not refresh the list of posts. However, we also want to refresh the list when a new post is created. Let's fix that now.

Invalidating queries

To ensure that the post list is refreshed after creating a new post, we need to invalidate the query. We can make use of the query client to do this. Let's do it now:

1. Edit `src/components/CreatePost.jsx` and import the `useQueryClient` hook:

```
import { useMutation, useQueryClient } from '@tanstack/react-query'
```

2. Use the query client to invalidate all queries starting with the `'posts'` query key. This will work with any query params to the `getPosts` request, as it matches all queries starting with `'posts'` in the array:

```
const queryClient = useQueryClient()
const createPostMutation = useMutation({
  mutationFn: () => createPost({ title, author, contents }),
  onSuccess: () => queryClient.invalidateQueries(['posts']),
})
```

Try creating a new post, and you will see that it works now, even with active filters and sorting! As we can see, TanStack Query is great for handling server state with ease.

Summary

In this chapter, we learned how to create a React frontend and integrate it with our backend using TanStack Query. We have covered the main functionality of our backend: listing posts with sorting, creating posts, and filtering by author. Dealing with tags and deleting and editing posts are similar to the already explained functionalities and are left as an exercise for you.

In the next chapter, [***Chapter 5**, *Deploying the Application with Docker and CI/CD**](#), we are going to deploy our application with Docker and set up CI/CD pipelines to automate the deployment of our application.

